

```

"errors"
"fmt"
"sync"
_ "github.com/mattn/go-sqlite3"
)
type ConnectionPool struct {
    queue      chan *sql.DB
    maxSize    int
    currentSize int
    lock       sync.Mutex
    isNotFull  *sync.Cond
    isNotEmpty *sync.Cond
}

func (cp *ConnectionPool) Get() (*sql.DB, error) {
    cp.lock.Lock()
    defer cp.lock.Unlock()

    // If queue is empty, wait
    for cp.currentSize == 0 {
        fmt.Println("Waiting for connection to be added back
in the pool")
        cp.isNotEmpty.Wait()
    }
    fmt.Println("Got connection!! Releasing")
    db := <-cp.queue
    cp.currentSize--
    cp.isNotFull.Signal()

    err := db.Ping()
    if err != nil {
        return nil, err
    }

    return db, nil
}

func (cp *ConnectionPool) Add(db *sql.DB) error {
    if db == nil {
        return errors.New("database not yet initiated. Please
create a new connection pool")
    }

    cp.lock.Lock()
    defer cp.lock.Unlock()

    for cp.currentSize == cp.maxSize {
        fmt.Println("Waiting for connection to be released")
        cp.isNotFull.Wait()
    }

    cp.queue <- db

    cp.currentSize++
    cp.isNotEmpty.Signal()

    return nil
}

func (cp *ConnectionPool) Close() {
    cp.lock.Lock()
    defer cp.lock.Unlock()

    for cp.currentSize > 0 {
        db := <-cp.queue
        db.Close()
        cp.currentSize--
        cp.isNotFull.Signal()
    }

    close(cp.queue)
}

func NewConnectionPool(driver, dataSource string, maxSize
int) (*ConnectionPool, error) {

    // Validate driver and data source
    _, err := sql.Open(driver, dataSource)
    if err != nil {
        return nil, err
    }

    cp := &ConnectionPool{
        queue:      make(chan *sql.DB, maxSize),
        maxSize:    maxSize,
        currentSize: 0,
    }

    cp.isNotEmpty = sync.NewCond(&cp.lock)
    cp.isNotFull = sync.NewCond(&cp.lock)

    for i := 0; i < maxSize; i++ {
        conn, err := sql.Open(driver, dataSource)
        if err != nil {
            return nil, err
        }
        cp.queue <- conn
        cp.currentSize++
    }

    return cp, nil
}

```

Of course, there are many ways in which this implementation can be improved upon. Typically, you can use